

Създаване и изпълнение на прости shell скриптове — 12б, Операционни системи

Увод

Shell скриптовете са последователности от команди, записани в текстов файл, които се изпълняват автоматично от командния интерпретатор (shell). Те позволяват автоматизация на повтарящи се задачи, ускоряват работата на системния администратор и осигуряват последователност при изпълнение на операции.

Защо да използваме shell скриптове?

- **Автоматизация** – изпълнение на серии от команди без ръчна намеса.
- **Проследимост** – скриптът служи като документация на процеса.
- **Повторяемост** – гарантирана еднаквост при всяко изпълнение.
- **Гъвкавост** – лесно адаптиране чрез променливи и условия.
- **Интеграция** – комбиниране на различни инструменти и команди.

1. Shell и shell скрипт

Какво представлява shell?

Shell е програмата, която приема командите от потребителя и ги изпълнява. Най-разпространените shell среди в Unix/Linux са:

- **bash** (Bourne Again Shell) – стандартният shell в повечето Linux дистрибуции.
- **sh** – оригиналният Bourne Shell.
- **zsh, fish, ksh** – алтернативни shell среди с допълнителни възможности.

Shell скрипт е текстов файл, съдържащ команди, които shell интерпретаторът изпълнява последователно.

2. Подготовка на работната среда

Какво ни е необходимо?

- **Текстов редактор** – например nano, vim, gedit или Visual Studio Code.
- **Терминал** – за изпълнение на скрипта (gnome-terminal, konsole и др.).
- **Право за изпълнение** – чрез командата chmod, тъй като скриптовете са файлове.
- **Shebang** – първият ред на скрипта, който указва кой интерпретатор да се използва (#!/bin/bash).

Работа в Windows среда

В класната стая работим на компютри с Windows, но също можем да създаваме и изпълняваме shell скриптове. Най-често се използват следните варианти:

- **Git Bash** – инсталира се заедно с [Git for Windows](#) и предоставя bash терминал. Отваряме Git Bash, навигираме до папката със скрипта и изпълняваме ./hello.sh.
- **Windows Subsystem for Linux (WSL)** – позволява стартиране на пълна Linux среда (Ubuntu, Debian и др.). Скриптовете се запазват в \\wsl\$ или директно от Linux файловата система и се изпълняват чрез wsl или в терминала на WSL.

- **PowerShell** – ако искаме да изпълняваме PowerShell скриптове, файловете имат разширение `.ps1` и синтаксисът е различен. За целите на урока използваме `bash`, затова препоръчваме Git Bash или WSL.

При записване на файловете използвайте редактор, който запазва текста в UTF-8 без BOM (например Notepad++, Visual Studio Code). Не забравяйте да зададете права за изпълнение чрез `chmod +x script.sh` в избраната Linux среда.

3. Създаване на първи shell скрипт

Стъпки за създаване на скрипт

1. Създай нов файл: `nano hello.sh`
2. Добави shebang на първия ред: `#!/bin/bash`
3. Напиши командите, които искаш да се изпълнят.
4. Запази файла и го направи изпълним: `chmod +x hello.sh`
5. Стартирай скрипта: `./hello.sh`

```
#!/bin/bash echo "Здравей, свят!" date pwd
```

Какво се случва при изпълнение?

- Shell интерпретаторът прочита файла ред по ред.
- Всяка команда се изпълнява в посочения ред.
- Резултатите се показват в терминала – съобщения, дати, текуща директория и т.н.

4. Shebang и права за изпълнение

Shebang (#!)

Първият ред в shell скрипта определя кой интерпретатор ще изпълни файла.

#!/bin/bash

Използва bash интерпретатор (най-често срещания).

#!/bin/sh

Съвместим с POSIX shell – гарантира работа на различни Unix системи.

#!/usr/bin/env bash

Намира bash динамично, подходящо за среди с различни пътеки.

#!/bin/zsh

Изпълнение чрез zsh – например за по-разширени функции.

Добавяне на права за изпълнение

По подразбиране нов файл няма право на изпълнение. За да го стартираме, трябва да използваме `chmod` :

```
chmod +x hello.sh
ls -l hello.sh
-rwxr-xr-x 1 user user 46 Nov 12 10:15 hello.sh
```

След това скриптът може да се изпълни с `./hello.sh` или `bash hello.sh`.

5. Структура и синтаксис на shell скриптовете

Основни елементи:

- **Коментари** – започват с # и не се изпълняват.
- **Команди** – стандартни Linux команди (ls , cp , grep).
- **Променливи** – съхраняват стойности, достъпват се с \$ (например \$USER).
- **Подмяна на команда** – използва се \$(команда) за записване на резултат.

Примерна структура

```
#!/bin/bash

# Автор: 12б клас
# Описание: Скрипт, който показва информация за системата

echo "Здравей, $USER!"
echo "Днешната дата е: $(date)"
echo "Текуща директория: $(pwd)"

FILES=$(ls | wc -l)
echo "Брой файлове: $FILES"
```

6. Управление на потока

Условни конструкции

В shell скриптовете можем да използваме if , case , цикли for и while .

```
# if условие
if [ -f "$1" ]; then
    echo "Файлът съществува."
else
    echo "Няма такъв файл."
fi
```

```
# for цикъл
for user in $(cut -d: -f1 /etc/pass
    echo "Потребител: $user"
done
```

```
# while цикъл
count=1
while [ $count -le 5 ]; do
    echo "Стъпка $count"
    count=$((count + 1))
done
```

```
# case конструкция
case "$1" in
    start) echo "Стартиране";;
    stop)  echo "Спиране";;
    *)     echo "Употреба: $0 {star
esac
```

7. Вход от потребителя и аргументи

Позиционни параметри

- \$0 – името на скрипта.
- \$1 , \$2 , ... – аргументи, подадени при стартиране.
- \$# – брой на аргументите.
- \$@ – всички аргументи като списък.

```
#!/bin/bash

if [ $# -lt 2 ]; then
    echo "Употреба: $0 <име> <възраст>"
    exit 1
fi

NAME=$1
AGE=$2

echo "Здравей, $NAME!"
echo "На $AGE години си."
```

Въвеждане по време на изпълнение

```
read -p "Въведи име на директория: " DIR

if [ -d "$DIR" ]; then
    echo "Директорията съществува."
else
    echo "Създавам директория $DIR"
    mkdir "$DIR"
fi
```

8. Практически мини-проекти

Резервно копие (backup)

```
#!/bin/bash

SRC="$HOME/Documents"
DEST="$HOME/backup_$(date +%F)"

mkdir -p "$DEST"
cp -r "$SRC" "$DEST"
echo "Резервно копие в $DEST"
```

Следене на дисковото пространство

```
#!/bin/bash

THRESHOLD=80
USAGE=$(df -h / | awk 'NR==2 {print $5}')

if [ "$USAGE" -gt "$THRESHOLD" ]; then
    echo "Внимание: използването е над 80%"
else
    echo "Нивата на диска са нормални"
fi
```

Меню в терминал

```
#!/bin/bash

while true; do
    echo "1) Показване на дата"
    echo "2) Показване на потребител"
    echo "3) Изход"
    read -p "Избери опция: " choice

    case $choice in
        1) date ;;
        2) whoami ;;
        3) echo "Довиждане!"; break ;;
        *) echo "Невалиден избор." ;;
    esac
done
```

9. Отстраняване на грешки и безопасно изпълнение

Полезни техники

- `bash -n script.sh` – проверка за синтактични грешки.
- `bash -x script.sh` – изпълнение с проследяване на командите.
- `set -e` – спиране при първа грешка.
- `set -u` – предупреждение при използване на неинициализирани променливи.

Сигурност

- Винаги проверявайте входните данни (`[ -f "$FILE" ]` , `[ -d "$DIR" ]`).
- Избягвайте изпълнение на команди, съставени от непроверени променливи.
- Използвайте кавички около променливи, за да предотвратите разделяне по интервали.
- Логвайте действията на скрипта (`echo` , `logger`).

10. Добри практики

- **Документация** – описвайте какво прави скриптът и кои са авторите.
- **Проверки** – валидирайте входни параметри и изходни кодове.
- **Функции** – разделяйте логиката на функции за по-добра четимост.
- **Логване** – записвайте важни събития и грешки.
- **Версионизиране** – съхранявайте скриптовете в системи като Git.
- **Портируемост** – използвайте POSIX съвместим синтаксис, когато е възможно.

Заклучение

Създаването на shell скриптове е ключово умение за всеки, който работи с операционни системи. Чрез тях можем да автоматизираме задачи, да намалим риска от грешки и да ускорим ежедневната работа. Започнете с прости скриптове, експериментирайте с условия и цикли и постепенно добавяйте нови функции.

Ключови изводи:

- **Shebang** указва интерпретатора и е необходим за изпълнение.
- **chmod +x** дава право за изпълнение на скрипта.
- **Променливите и аргументите** правят скриптовете гъвкави.
- **Условните конструкции и циклите** позволяват по-сложна логика.
- **Отстраняване на грешки** е важна част от процеса (използвайте `bash -x`).
- **Добри практики** гарантират безопасност и поддръжка на скрипта.

Практически съвет: Създайте собствена папка `~/scripts`, добавете я към `PATH`, запазвайте там полезни скриптове и ги споделяйте с екипа си. Така ще развиете навик да документирате и организирате автоматизациите си.